

Unit 3: Web and Service Application Architectures

Introduction. Web Application Architectures. Service Application Architectures. Cloud Computing.

Unit 3: Web and Service Application Architectures

- ▶ Introduction
 - ◆ Software Architecture
 - ◆ Architectural Views
- ▶ Web Application Architectures
- ▶ Service Application Architectures
- ▶ Cloud Computing

Software Architecture

Software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.

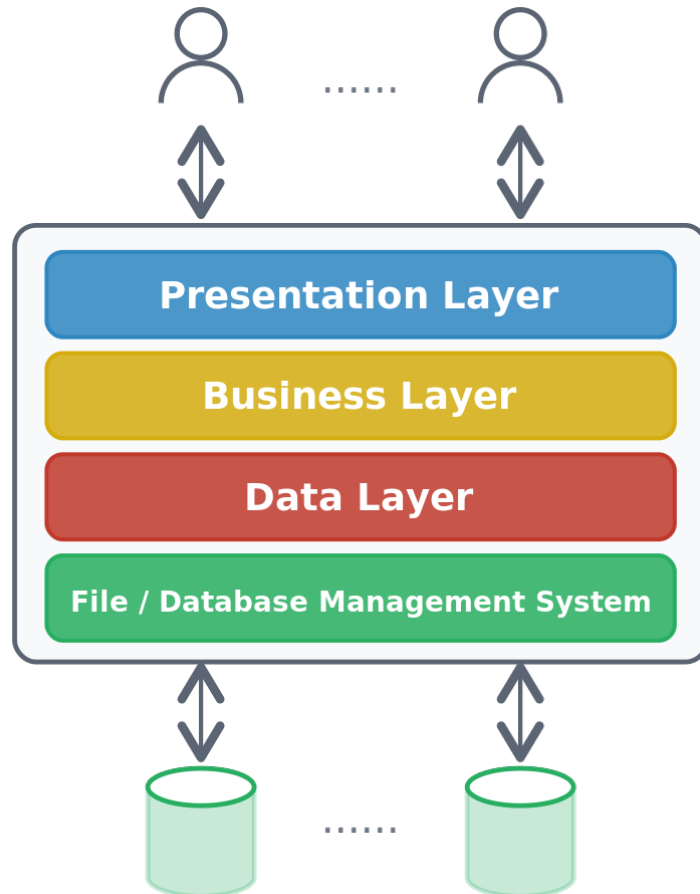


Architectural Viewpoints

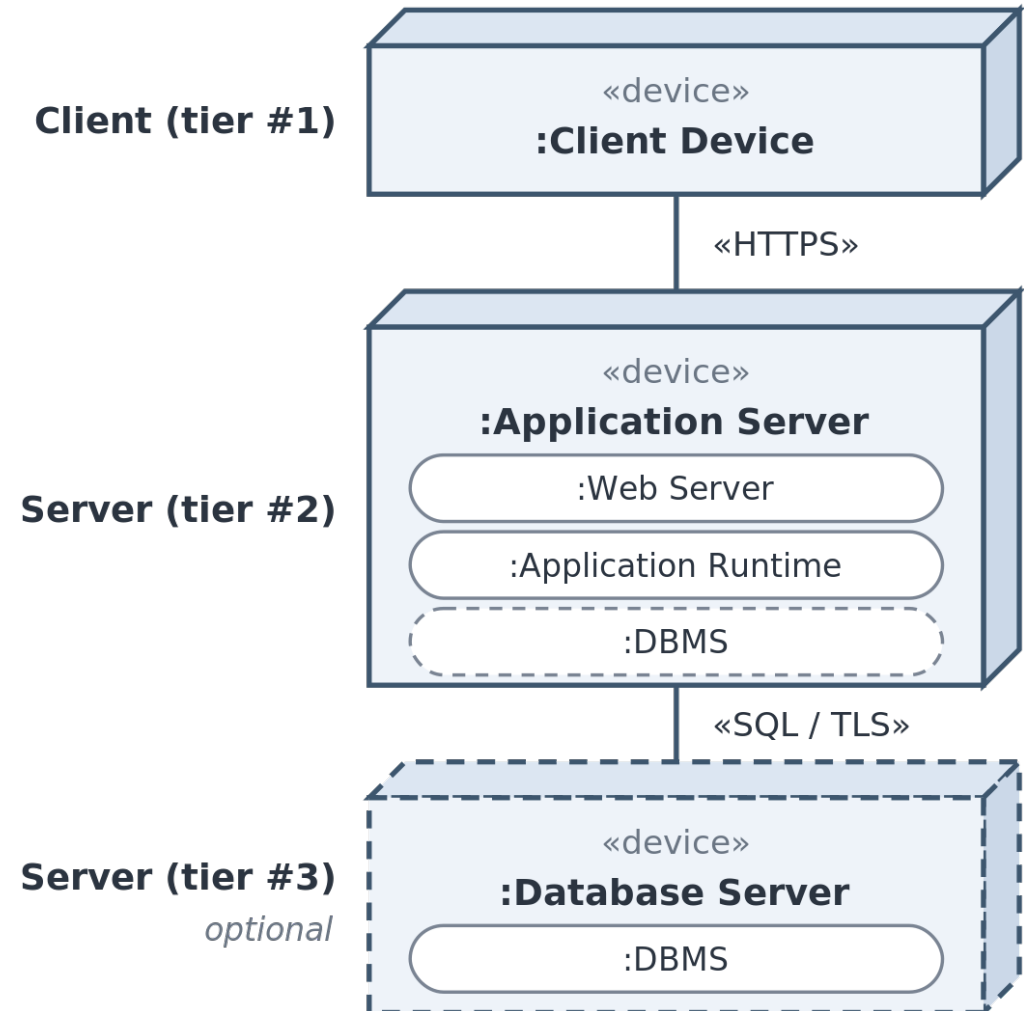
- ▶ **Logical view** is the large-scale organization of the software components into packages, subsystems and layers that logically separate the functionality of a software system. It is called logical architecture because there is no decision about how these elements are deployed across different computational nodes.
- ▶ **Physical view** is the organization and distribution of the logical architecture across different computational nodes in a network.

Architectural Viewpoints

3-Layered Logical Architecture



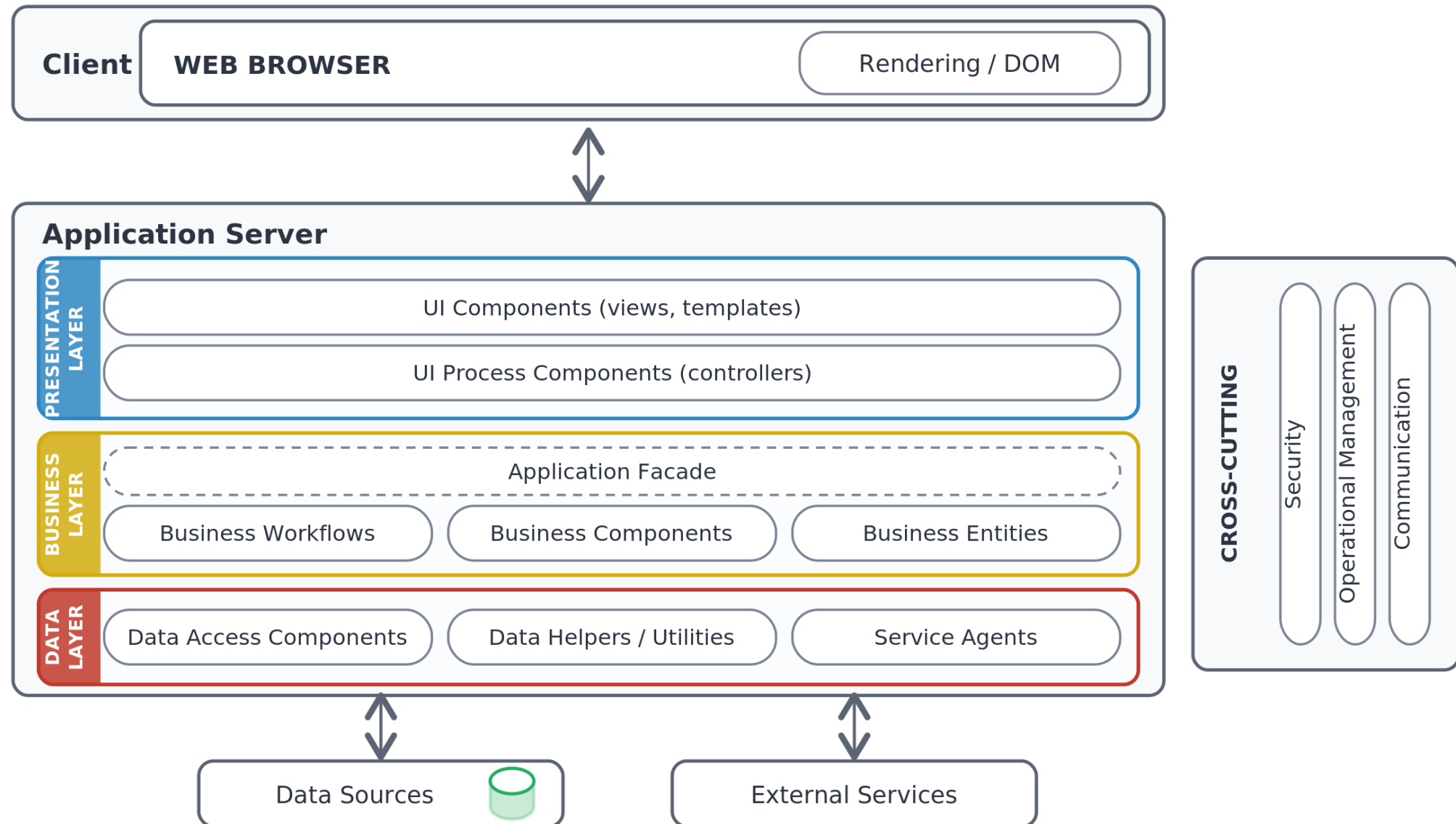
2(+)-Tiered Client/Server Physical Architecture



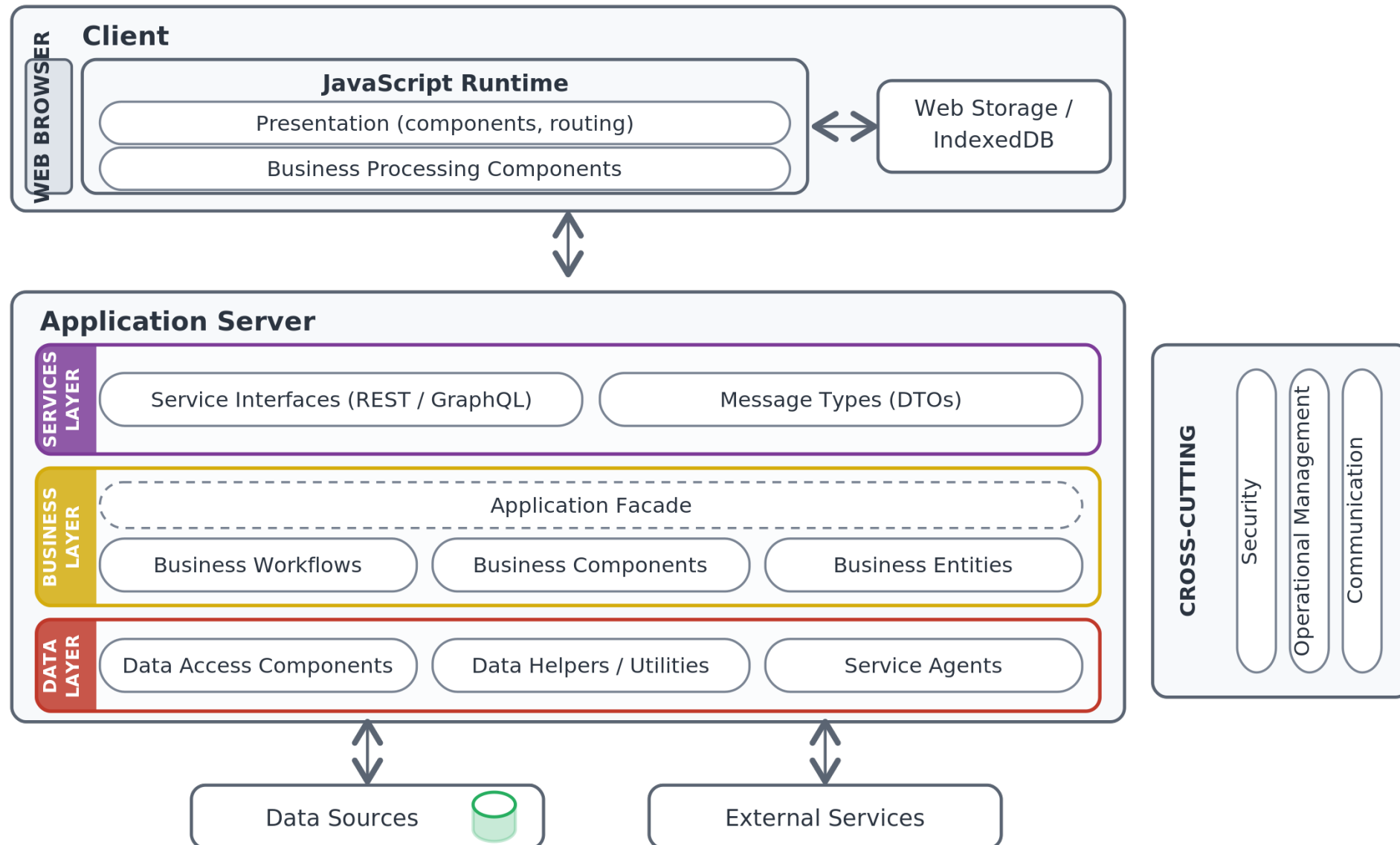
Unit 3: Web and Service Application Architectures

- ▶ Introduction
- ▶ Web Application Architectures
 - ◆ Logical - Physical Architecture
 - ◆ Dimensions to consider in a Physical Architecture Design
 - ◆ Architecture Patterns
 - ❖ Single server
 - ❖ Separate database
 - ❖ Replicated Web servers
 - ❖ Separated static and dynamic content
 - ◆ Web Caching
- ▶ Service Application Architectures
- ▶ Cloud Computing

Logical-Physical Architecture: “Classic” Web Apps



Logical-Physical Architecture: Single-Page Apps



Dimensions in Physical Architecture Design

- ▶ **Non-functional requirements** that pursue the achievement of an adequate level of service
- ▶ Physical, financial and organizational **constraints** that may affect decision-making
- ▶ Alternative **scenarios** in architecture deployment

Non-functional requirements

- ▶ Performance: The app must sustain the expected workload defined in terms of
 - ◆ the maximum number of concurrent users,
 - ◆ the number of page requests served per unit of time,
 - ◆ the maximum time for delivering a page to the client
- ▶ Scalability: The architecture must be easily extensible, ideally by adding more instances (horizontal scaling)
- ▶ Availability: Faults should not affect significantly the service delivered to users
- ▶ State maintenance (reliability): The state of the user interaction must be preserved, even when the application is distributed on multiple machines or failures occur
- ▶ Security:
 - ◆ Data should be protected
 - ◆ Users should be identified and granted access only to the data and functions they are entitled to

Constraints of Architecture Design

► Cost

- ◆ Every configuration requires a different investment, in terms of processors, network infrastructure, interfaces, and software licenses or cloud service subscriptions
- ◆ The application budget may limit the choice of hardware resources and software products

► Complexity

- ◆ Some configurations are simpler than others to set up and maintain
- ◆ The unavailability or the cost of specialized technical skills may constrain the architecture design

► Corporate standards and infrastructures

- ◆ The WebApp may be deployed within a corporate IT infrastructure, which may constrain the selection of hardware resources and software products.

Scenarios of Architecture Deployment

▶ On-premises

- ◆ The application runs in the organisation's own data centre and is operated by its internal IT department.

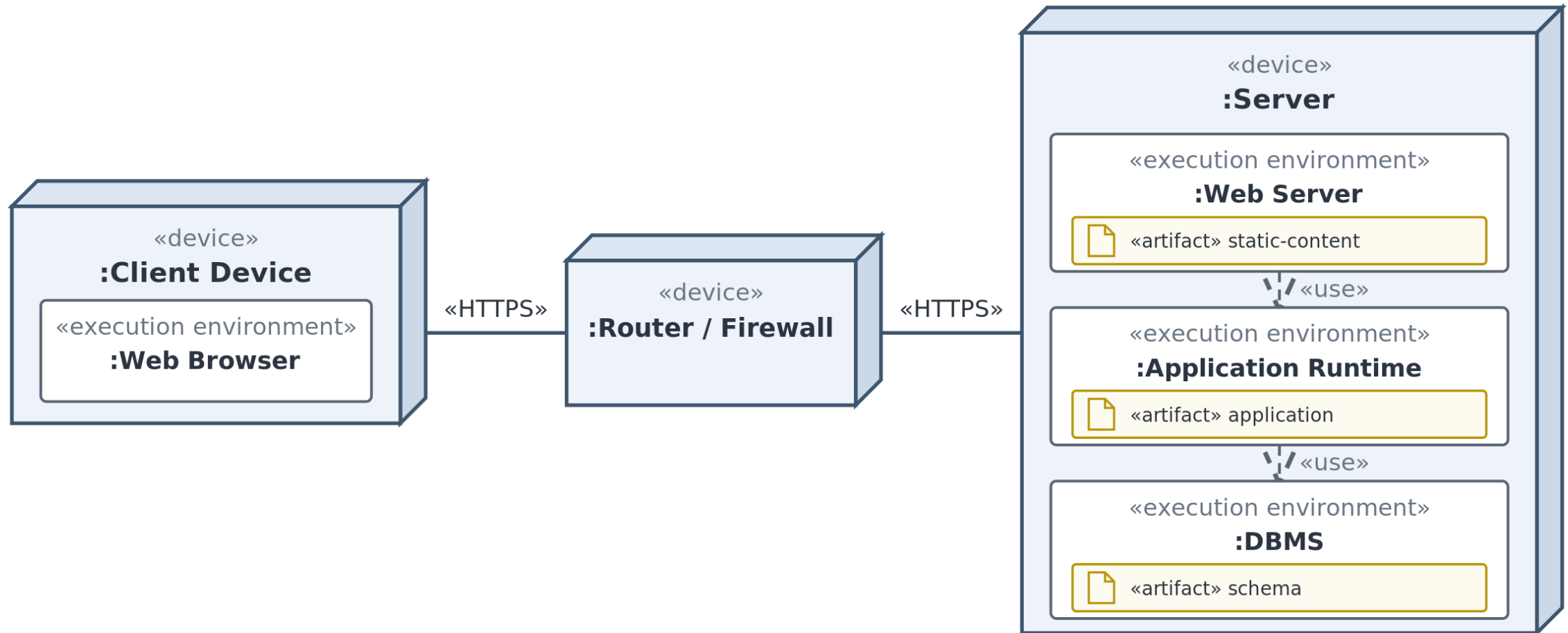
▶ Co-located

- ◆ The organisation's own equipment is installed in a third-party data centre, but it is still operated by its internal IT department.

▶ Cloud

- ◆ Infrastructure and its operation are provided by an external provider and provisioned on demand (IaaS or PaaS). The application is usually packaged as containers.

Architecture Pattern: Single Server



Single Server Pattern: Evaluation (1/2)

► Performance

- ◆ Depends on the configuration of the server: CPU speed, available memory, disk access latency, etc.
- ◆ The DBMS is both memory and CPU-intensive

► Scalability

- ◆ Is bound by the hardware architecture of the selected server

► Availability

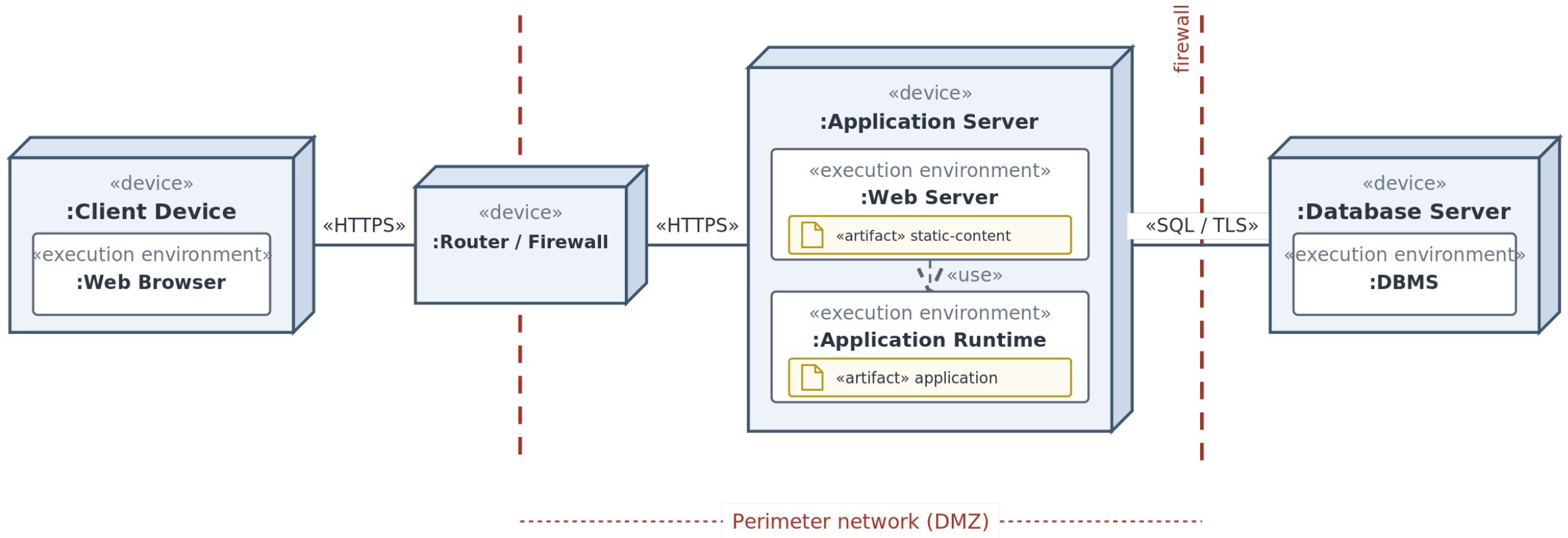
- ◆ Every software and hardware element is a single point of failure: if it breaks, the entire system hangs.
- ◆ Can be improved by adding redundant hardware resources (multiple CPUs, mirrored disks) and by installing multiple processes running different instances of the Web server, application runtime, and database

...

Single Server Pattern: Evaluation (2/2)

- ▶ State maintenance
 - ◆ No problems with none of the three possibilities to store user data: client, server or database
- ▶ Security
 - ◆ A weak aspect of this configuration: attackers breaking the firewall and the Web server can take control of the host and gain direct access to the database, violating data protection
- ▶ Low cost, as far as massive parallelism is not required.
- ▶ Low complexity

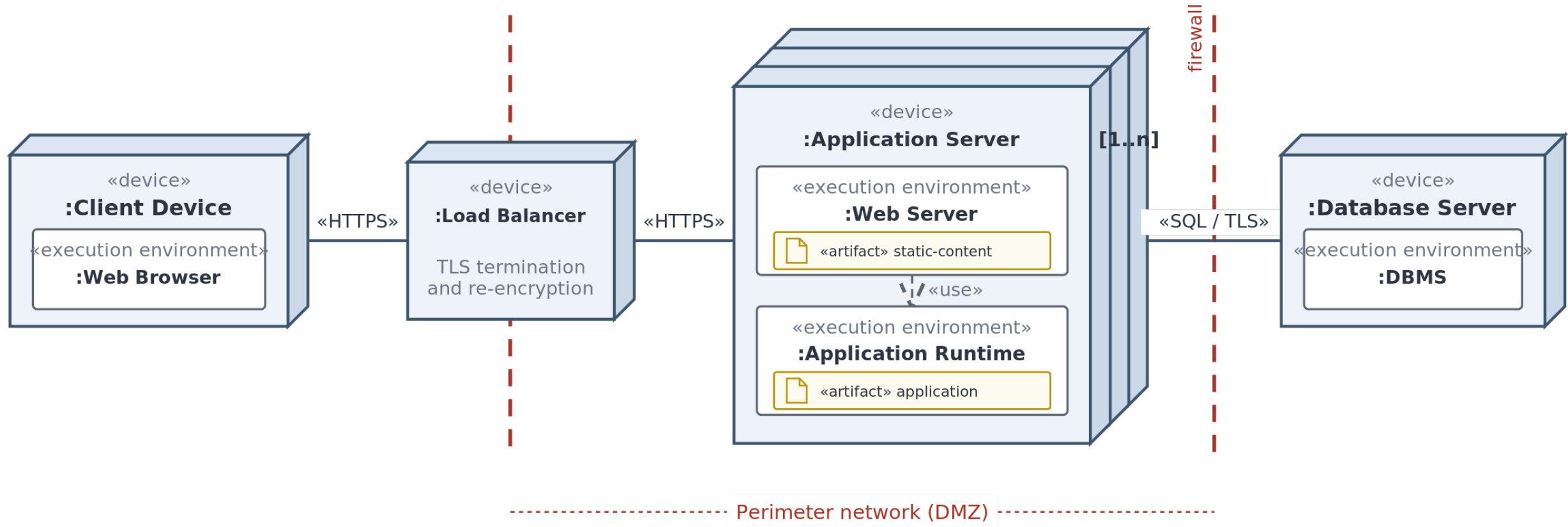
Architecture Pattern: Separate Database



Separate Database Pattern: Evaluation

- ▶ Better performance
 - ◆ One extra machine
 - ◆ Each tier can be tuned to the requirements of the installed software
- ▶ More scalable
 - ◆ It is possible to act separately on each tier.
 - ◆ Normally, the first bottleneck is in the middle tier
- ▶ Availability is not improved
 - ◆ Each component is still a single point of failure
- ▶ Significantly improved security
 - ◆ The inner firewall may disallow HTTPS requests at all and let only database requests pass, making it more difficult for attackers to reach the data tier.

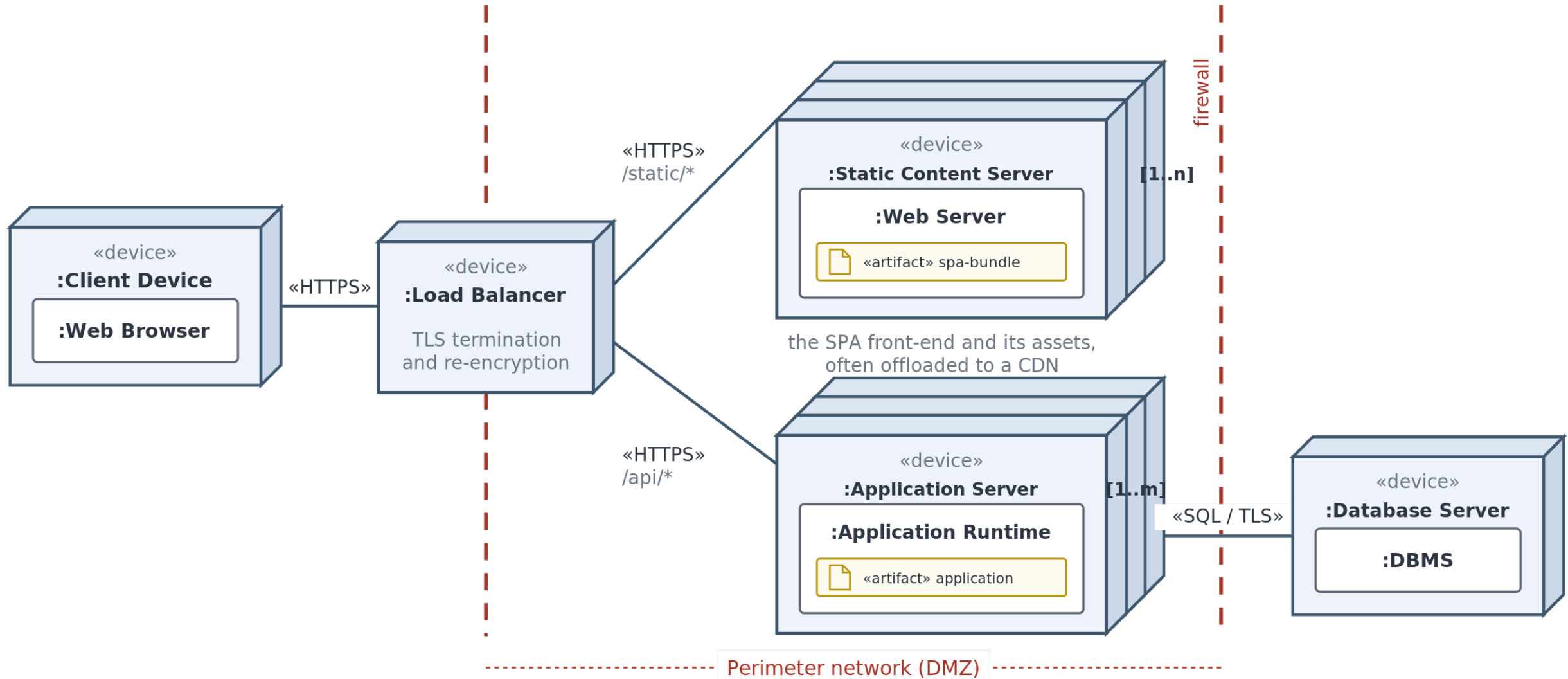
Architecture Pattern: Replicated Web Server



Replicated Web Server Pattern: Evaluation

- ▶ Improved performance and scalability:
 - ◆ Load balancing
 - ◆ Clustering: a group of servers (aka nodes) that provide a unified view of the services they individually offer.
- ▶ Improved availability:
 - ◆ Fail-over: if a cluster node fails, its workload can be redistributed to the other nodes of the same cluster
- ▶ Session state maintenance on the replicated servers
 - ◆ Session affinity (aka sticky sessions): The load balancer sends all the incoming requests pertinent to a given session to the same server.
 - ◆ Session migration: Session state is shared by the servers in the cluster
- ▶ Improved data transmission security
 - ◆ TLS is terminated at the load balancer and re-encrypted towards the servers.

Architecture Pattern: Separate Static and Dynamic Content



Separate Static and Dynamic Content Pattern: Evaluation

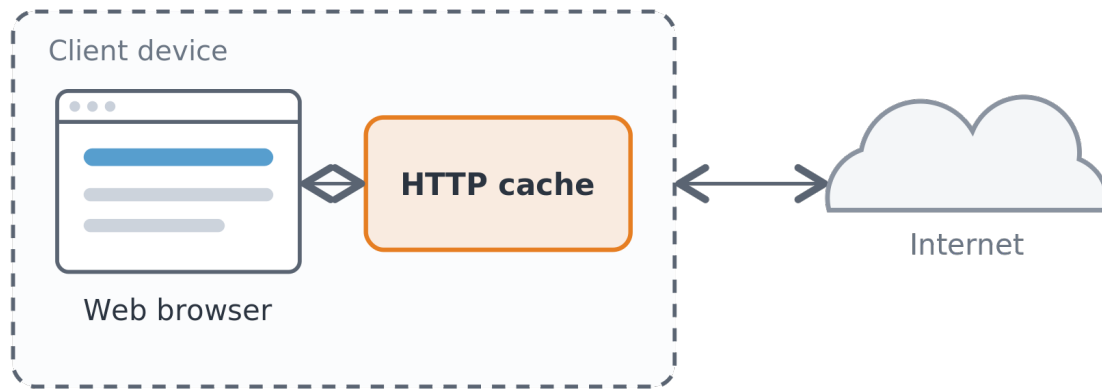
- ▶ Improved performance, scalability, and availability
 - ◆ The static tier and the dynamic tier can be replicated independently so that the number and configuration of the hosts can be optimized:
 - ❖ a well-balanced configuration may require more instances for the dynamic tier than for the static one.
 - ◆ Static assets are usually pushed out to a CDN, which removes that traffic from the origin altogether.
- ▶ The additional cost is operational: two deployment units to build, release, and monitor.

Web Caching

- ▶ Caching consists of temporarily storing resources in a fast access location, for later retrieval.
- ▶ Benefits:
 - ◆ Reduction of the response time
 - ◆ Reduction of computation effort when the resource is dynamically built
- ▶ Many things can be cached:
 - ◆ Static HTML pages and multimedia files.
 - ◆ Fragments of pages computed by the application.
 - ◆ Intermediate data consumed by the application for producing pages, e.g. JSON documents.
 - ◆ The result of database queries or other application commands

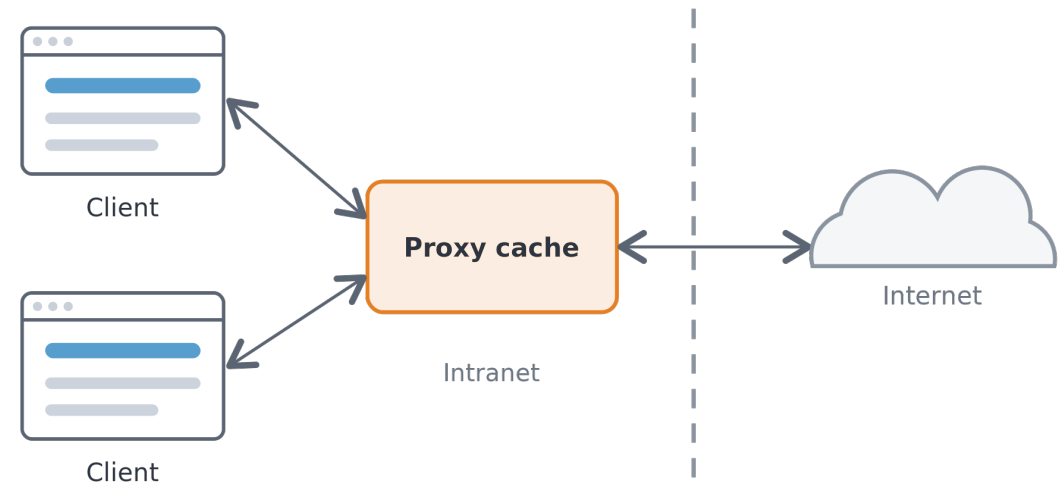
Web caching: Where to Cache (1/3)

Browser caching



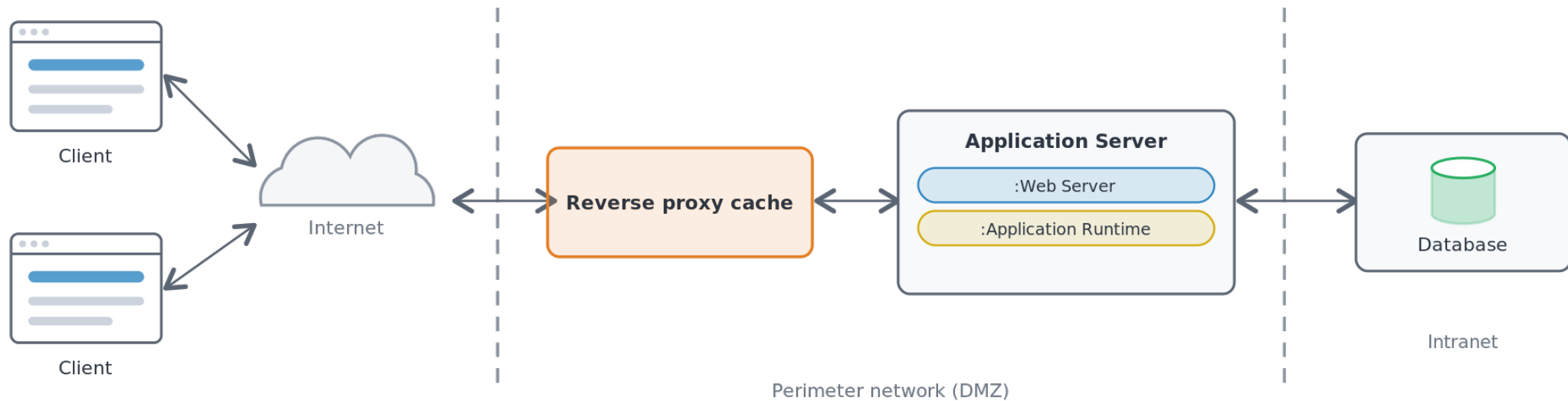
Every Web browser contains a cache of HTML pages and multimedia files used to speed up the rendition of pages that contain cached objects.

Proxy caching



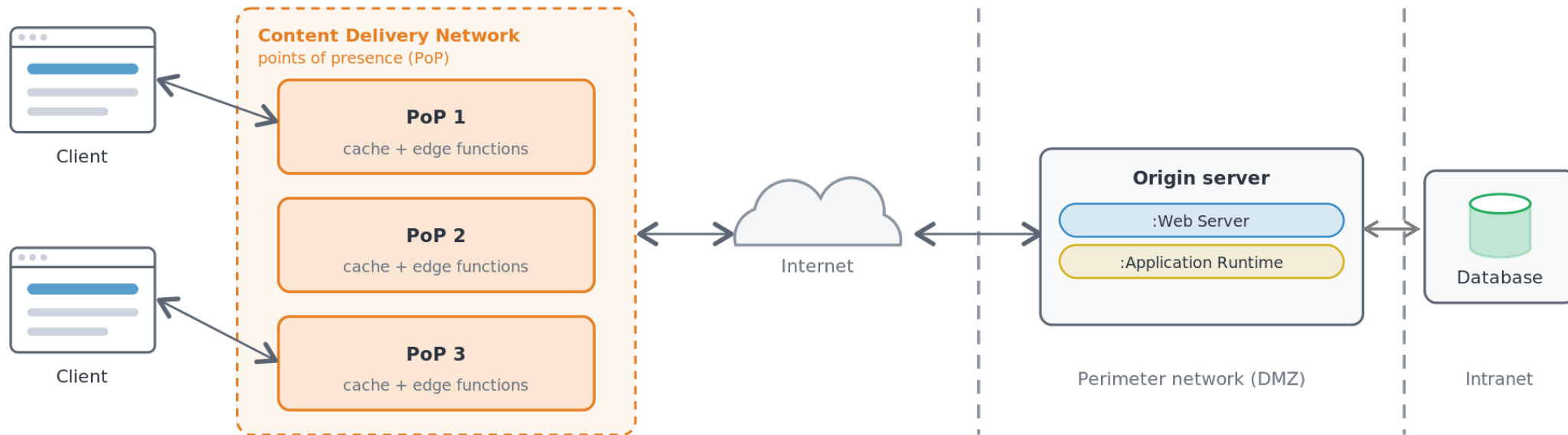
Proxy caches store a local copy of the resources requested by users, avoiding Internet access for frequently asked pages. They are rare today, because traffic is encrypted with HTTPS

Web caching: Where to Cache (2/3): **Reverse proxy caches**



- ▶ A reverse proxy cache is placed in front of a server cluster: it intercepts requests, caches copies of the objects produced by the servers, and delivers them to subsequent requests.
- ▶ Page prefetching: based on the last request, the reverse proxy loads into cache the pages that are more likely to be requested next.
- ▶ In front of the servers: Varnish, NGINX, Apache Traffic Server.
- ▶ Beside the application, for query results and session data: Redis, Memcached.

Web caching: Where to Cache (3/3): Content delivery networks (CDNs)

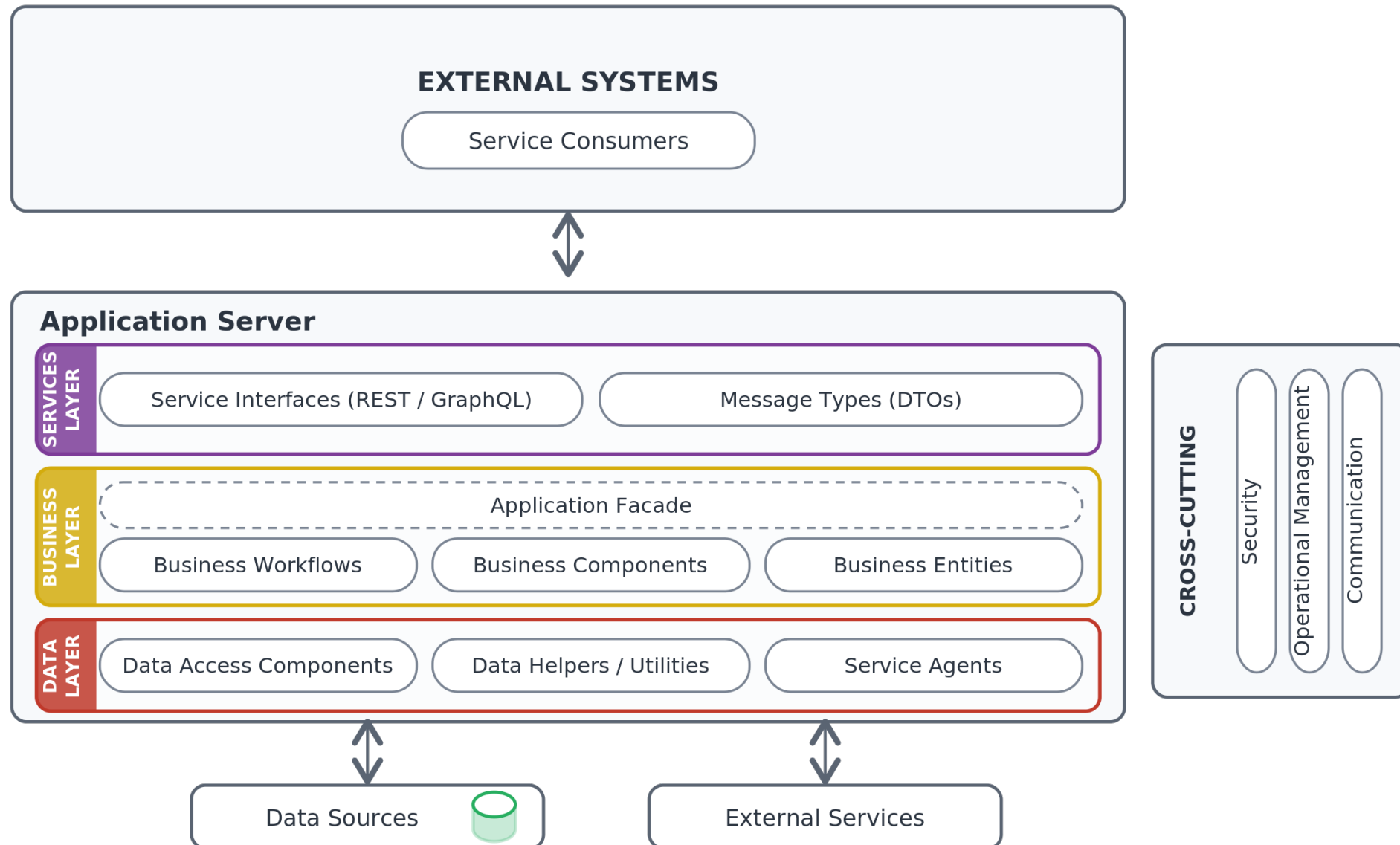


- ▶ CDNs allow content providers to outsource their caching infrastructure to a network of servers distributed across the Internet
- ▶ Requests are routed to the point of presence (PoP) closest to the client by DNS resolution or anycast routing
- ▶ The CDN serves the request from the optimal copy, and may also run application code at the edge (edge functions).
- ▶ Major providers: Akamai, Cloudflare, Amazon CloudFront, Fastly.

Unit 3: Web and Service Application Architectures

- ▶ Introduction
- ▶ Web Application Architectures
- ▶ Service Application Architectures
 - ◆ Logical - Physical Architecture
 - ◆ Dimensions to consider in a Physical Architecture Design
 - ◆ Architecture Patterns
- ▶ Cloud Computing

Logical-Physical Architecture: Service Apps



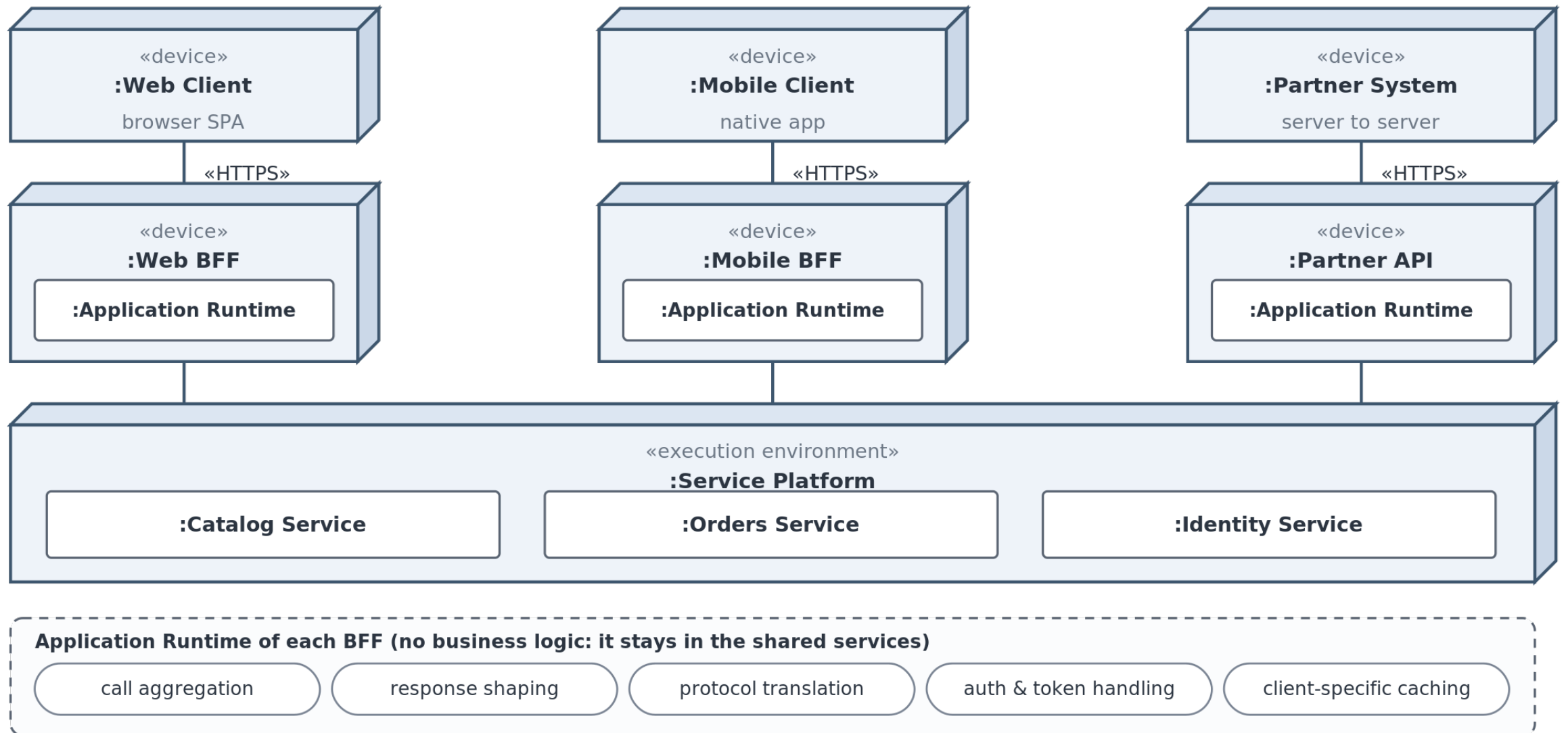
Dimension in Physical Architecture Design

- ▶ **Non-functional requirements, constraints** and alternative **scenarios** in architecture deployment are the same as for Web applications

Architecture Patterns

- ▶ Service applications may be deployed using the same architectural patterns as for Web applications considering the performance and security issues inherent in distributed scenarios
- ▶ The static content tier is not needed when the application only exposes an API: every request goes to the application tier.
- ▶ When several kinds of clients consume the services, the *Backend for Frontend* pattern adds one tier per client experience.

Architecture Pattern: Backend for Frontend (BFF)



Backend for Frontend Pattern: Evaluation

▶ Tailored APIs

- ◆ Each client gets a response shaped to its needs: no over-fetching, no under-fetching, and no single API trying to satisfy conflicting requirements.

▶ Independent evolution

- ◆ Each BFF is owned, deployed, and scaled by the team that owns its frontend.

▶ Fault isolation

- ◆ An outage or an overload of one BFF does not affect the other clients.

▶ Costs and risks

- ◆ More services to build, deploy, monitor, and secure, and logic that may be duplicated across BFFs.
- ◆ A BFF must stay thin: the business logic belongs to the shared downstream services.

Unit 3: Web and Service Application Architectures

- ▶ Introduction
- ▶ Web Application Architectures
- ▶ Service Application Architectures
- ▶ Cloud Computing

Cloud Computing – A Definition

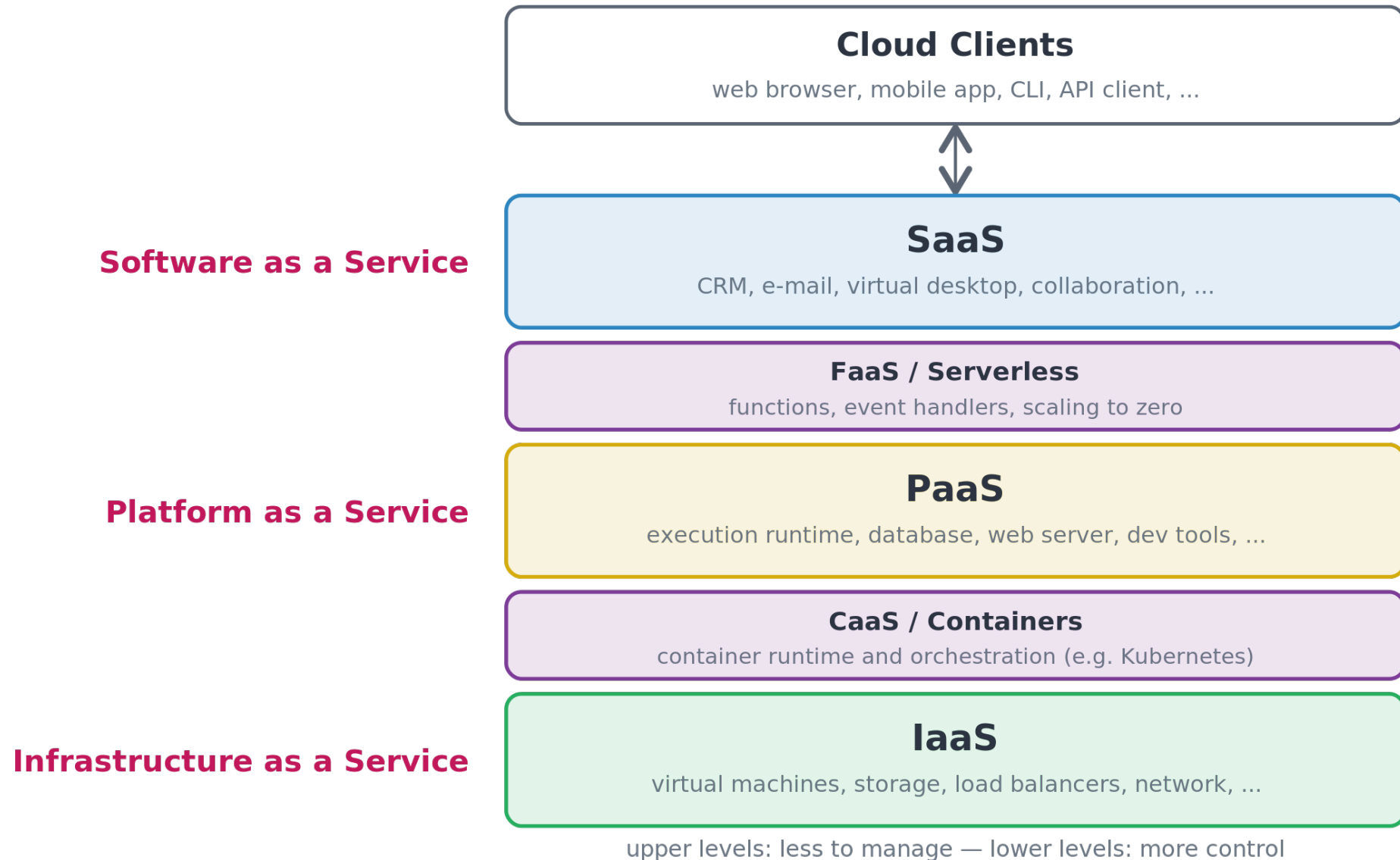
“Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model is composed of five essential characteristics, three service models, and four deployment models”

The NIST Definition of Cloud Computing

Cloud Computing – 5 Essential Characteristics

- ▶ **On-demand self-service**: A consumer can unilaterally provision computing capabilities as needed, such as server time and network storage, as needed automatically without requiring human interaction
- ▶ **Broad network access**: Capabilities are available over the network and accessed through standard mechanisms.
- ▶ **Resource pooling**: Physical and virtual resources are dynamically assigned and reassigned according to consumer demand.
- ▶ **Rapid elasticity**: Capabilities can be rapidly and elastically provisioned.
- ▶ **Measured Service**: Resource usage is monitored, controlled, and reported providing transparency for both the provider and consumer of the utilized service.

Cloud Computing – Service Models



Cloud Computing – Deployment Models

- ▶ **Private cloud**: provisioned for exclusive use by a single organization
- ▶ **Community cloud**: provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns
- ▶ **Public cloud**: provisioned for open use by the general public.
- ▶ **Hybrid cloud**: composition of two or more distinct cloud infrastructures (private, community, or public) bound together by standardized or proprietary technology that enables data and application portability

References

- ▶ S. Ceri et al. *Designing Data-Intensive Web Applications*. Chapter 10. Morgan Kaufmann, 2003.
- ▶ Microsoft. *Azure Architecture Center: application architecture fundamentals*. <https://learn.microsoft.com/azure/architecture/guide/>
- ▶ National Institute of Standards and Technology. *The NIST Definition of Cloud Computing*. SP 800-145, 2011.
<https://csrc.nist.gov/pubs/sp/800/145/final>